

Consider the following class declaration for representing cards to be used in a program that simulates a card game.

```
public class Card implements Comparable<Card>
{
    // postcondition: returns the value of this card
    public int value();

    // postcondition: returns the value 0 if the argument Card is equal to this Card;
    //               a value less than 0 if this Card is less than the Card argument;
    //               and a value greater than 0 if this Card is greater than the Card
    //               argument.
    public int compareTo(Card c);

    // postcondition: returns true if this Card has the same rank as the specified
    //               object obj.
    public boolean equals(Object obj);

    // Constructors and other methods not shown
}
```

The card game to be simulated is a two-player game called “Compete” that is played with a set of cards, each having a numeric value. The object of the game is to win all the cards.

Each player has a pile of cards and both piles start with the same number of cards. Play consists of a sequence of “rounds”. Play continues until at the end of a round one player is out of cards. A discard pile is created and used during the rounds and is empty at the beginning of each round.

You will write code to move cards from one pile to another and to play one round. Each pile of cards will be represented by a queue.

A round is played by executing the following steps until the round ends.

1. If exactly one player’s pile is empty, the other player adds all the cards in the discard pile to the bottom of his or her pile without changing the order. The round ends.

Each player turns over the top card from his/her pile.

2. If the cards match in value, both cards are added to the discard pile in either order and play returns to step 1.
3. If the cards do not match in value, the player whose card has the greater value adds any cards in the discard pile to the bottom of his/her pile without changing the order of the cards in the discard pile. He/she then adds both cards just played to the bottom of his/her pile in either order. The round ends.

- (a) Write the method **appendQueue**, which is described as follows. **appendQueue** should remove all the cards from the argument source and add them to the argument destination in the same order.

```
// precondition: all cards have been removed from source
//                and added to destination in the same order
public void appendQueue(Queue<Card> destination, Queue<Card> source) {

}
```

- (b) You will write function **oneRound**. **oneRound** takes the two players' piles of cards as parameters and carries out one round of the game. For your convenience the steps for one round are repeated here.
1. If exactly one player's pile is empty, the other player adds all the cards in the discard pile to the bottom of his or her pile without changing the order. The round ends.

Each player turns over the top card from his/her pile.

2. If the cards match in value, both cards are added to the discard pile in either order and play returns to step 1.
3. If the cards do not match in value, the player whose card has the greater value adds any cards in the discard pile to the bottom of his/her pile without changing the order of the cards in the discard pile. He/she then adds both cards just played to the bottom of his/her pile in either order. The round ends.

In writing **oneRound**, you may call any of the methods in the Card class and **appendQueue** from part (a).

Complete the method **oneRound**.

```
// precondition: pile1.length() > 0; pile2.length() > 0
// precondition: pile1 and pile2 have been updated according to the
//                rules of the game
public void oneRound(Queue<Card> pile1, Queue<Card> pile2) {

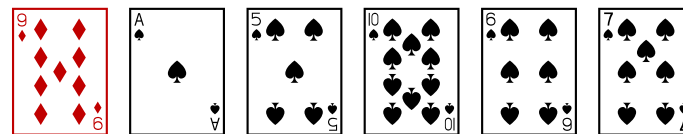
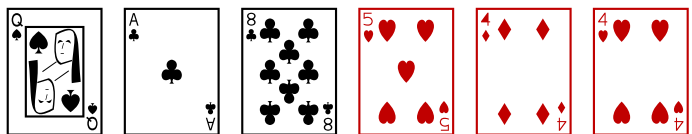
}
```

Sample runs:

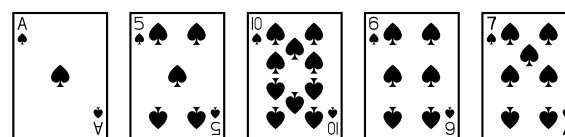
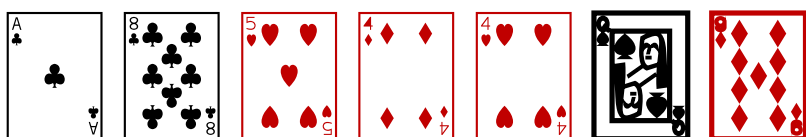
Player 1

Original Hands

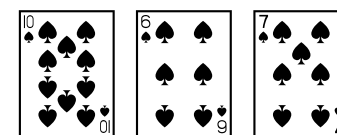
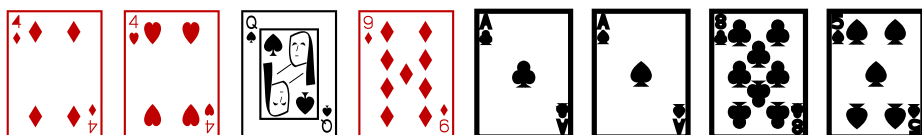
Player 2



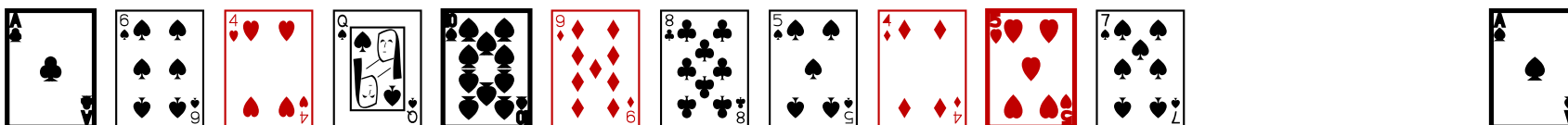
Result of Round 1 - Q_{spades} beats 9_{diamonds} (rule 2)



Result of Round 2 - A_{clubs} ties A_{spades} (rule 3) then 8_{clubs} beats 5_{spades} (rule 2)



Example of why Rule 1 is necessary. Let's say after much play these are the players' hands:



Then both aces would be played, but rule 3 doesn't end the round. Rule 2 wouldn't apply since player 2 has no cards. Rule 1 would mean Player one picks up the aces from the discard pile and wins the game.

